Bully Election Code Report:

Nama: **Ananda Aryasatya Zhafran Aqila**
NRP: **5025241142**

Penjelasan

Bully Election adalah mekanisme coordinator election yang bertahap dari nodeID paling kecil ke paling besar. Ketika sebuah node mengetahui bahwa coordinator saat ini tidak hidup, maka ia akan memberikan perintah eleksi ke node dengan ID yang lebih tinggi. Setelah node dengan ID lebih tinggi mendapatkan perintah eleksi, maka node ID lebih tinggi tersebut akan melakukan eleksi juga ke node dengan ID yang lebih tinggi lagi, dan akan mengembalikan “OK” ke node pengirim (node dengan ID lebih rendah). Ketika mendapak response “OK”, maka si pengirim tahu bahwa ia bukanlah node dengan ID tertinggi, sehingga node tersebut tidak akan menjadi coordinator. Hal ini akan dilakukan berulang kali hingga, tidak ada node yang lebih tinggi yang mengirimkan “OK” (atau bisa dibilang node pengirim adalah node dengan ID tertinggi yang sedang hidup). Ketika hal tersebut tercapai, maka node yang tidak mendapat “OK” inilah yang menjadi coordinator.

The code:

```go
package main

import (
        "fmt"
        "time"
        "strings"
        "os"
        "strconv"

        "go.nanomsg.org/mangos/v3"
        "go.nanomsg.org/mangos/v3/protocol/rep" // Example: Reply protocol
socket
        "go.nanomsg.org/mangos/v3/protocol/req"
        _ "go.nanomsg.org/mangos/v3/transport/tcp"
)

var NODEID int;
var CURRENTCOORDINATOR = 7;
var BASEPORT = 5000
var MAXNODEID = 7;
```

```
func nodePrintf(format string, args ...any) {
	fmt.Printf("[ node %d] "+format+"\n", append([]any{NODEID},
args...)...)
}


func sendToNode(targetIp string,targetPort string, nodeID int, message string)
(string,error) {
	var sock mangos.Socket
	var err error

	if sock, err = req.NewSocket(); err != nil {
		return "",err
	}
	defer sock.Close()

	// Dial to the target node's address
	addr := "tcp://127.0.0.1:" + targetPort
	if err = sock.Dial(addr); err != nil {
		return "", err
	}

	// Set a small write/send timeout so it doesn't hang forever if the
node is dead
	sock.SetOption(mangos.OptionSendDeadline, 1*time.Second)

	// Send message
	if err = sock.Send([]byte(buildMessage(nodeID, message))); err != nil {
		return "", err
	}

	// Wait for reply (like "OK")
	reply, err := sock.Recv()
	if err != nil {
		return "", err // If target node is dead (Node 7), this will
trigger an error/timeout!
	}
	a, _ := strconv.Atoi(targetPort)

	nodePrintf("Got reply from %d: %s", a-BASEPORT, string(reply))
	return string(reply), nil
}
```

```
func buildMessage(node_id int, msg string) string {
    return fmt.Sprintf("%d:%s", node_id, msg)
}

func parseMessage(msg string) (string, []string) {
    parts := strings.Split(msg, ":")
    if len(parts) < 2 {
        return "", []string{}
    }
    nodeFromStr := parts[0]
    actualMsg := parts[1:] // Returns slice like ["ELECTION"] or
["COORDINATOR", "6"]
    return nodeFromStr, actualMsg
}

func startElection(nodeID int) {
        got_OK := false
        for i := nodeID + 1; i <= MAXNODEID; i++ {
                // EXPLANATION::: lets try to send to the higher nodes, is
there any?
        portStr := fmt.Sprintf("%d", 5000+i)
                repl, err := sendToNode("127.0.0.1", portStr, NODEID,
"ELECTION")
                if repl == "OK" && err == nil {
                        got_OK = true;
                }
    }

        if got_OK {
                // EXPLANATION::: got OK, so, there are/is higher node ID than
me, ok then
                return
        } else {

                // if there is no another OK, then this node is the highest
node
                for i := 0; i <= MAXNODEID; i++ {
                        if i == NODEID {
                                continue
                        }
                        repl, _ := sendToNode("127.0.0.1", fmt.Sprintf("%d",
5000+i), NODEID, fmt.Sprintf("COORDINATOR:%d", NODEID))
                        if repl == "OK" {
                                got_OK = true;
                        }
                }
```

```
        }

}


func main() {
        time.Sleep(2 * time.Second)
        if len(os.Args) < 3 {
                nodeID := "?"
                if len(os.Args) > 1 {
                        nodeID = os.Args[1]
                }
                fmt.Printf("[ node %s] Usage: ./bully <node_id> <state>\n",
nodeID)
                fmt.Printf("[ node %s] Example: ./bully 4 live_init\n", nodeID)
                os.Exit(1)
        }
nodeIDstr := os.Args[1]
    nodeIDchange, err := strconv.Atoi(nodeIDstr) // First time err is declared
in main
    if err != nil {
        fmt.Printf("[ node %s] Invalid Node ID: must be an integer\n",
nodeIDstr)
        os.Exit(1)
    }
    NODEID = nodeIDchange

    state := os.Args[2]
    nodePrintf("Starting up with state: %s", state)

    switch state {
    case "dead":
        os.Exit(1)
                return
    case "live_init":
                time.Sleep(3 * time.Second)
        startElection(NODEID)
    }

    ip := "127.0.0.1"
    port := fmt.Sprintf("%d", BASEPORT+NODEID)
    ip_port := ip + ":" + port

    var sock mangos.Socket

    // Use '=' instead of ':=' because err already exists in this function
```

```
block
    sock, err = rep.NewSocket()
    if err != nil {
        nodePrintf("Can't get new rep socket: %s", err)
        os.Exit(1)
    }

    // Use '=' for err here too
    err = sock.Listen("tcp://" + ip_port)
    if err != nil {
        nodePrintf("Can't listen on socket: %s", err)
        os.Exit(1)
    }

    nodePrintf("Node listening on %s...", ip_port)

        // 3. Background loop to continuously receive messages
        go func() {
                for {
                        // Recv blocks until a message arrives from a peer
                        msg, err := sock.Recv()
                        if err != nil {
                                nodePrintf("Error receiving: %s", err)
                                break
                        }

                        nodePrintf("Received message: %s", string(msg))

                        node_from_str, actual_msg := parseMessage(string(msg))
                        node_from, err := strconv.Atoi(node_from_str)

                        switch actual_msg[0] {
                        case "ELECTION":
                                if node_from < NODEID {
                                        // EXPLANATION::: the node id where the
message from is lower than me
                                        sock.Send([]byte("OK"))

                                        // EXPLANATION::: so, send the ok, then
let's see is there any higher ID node?
                                        go startElection(NODEID) // Kick off
your election routine
                                } else {
                                        // EXPLANATION::: the node id where the
message from is higher than me
                                        // EXPLANATION::: just need to send the
```

```
ok, because I will never be the COORDINATOR
                                        sock.Send([]byte("OK"))
                                }

                        case "COORDINATOR":
                                // EXPLANATION::: the new coordinator has been
elected, so, let's update the data.
                                new_coordinator_int, _ :=
strconv.Atoi(actual_msg[1])
                                CURRENTCOORDINATOR = new_coordinator_int
                                nodePrintf("NEW COORDINATOR: %d",
new_coordinator_int)
                                sock.Send([]byte("OK"))
                        }

                }
        }()

        // Keep main running
        select {}
}
```

pada kode ini, saya membuat sistemnya sama dengan penjelasan di atas. Terdapat 2 bagian utama kode: `startElection` yang bertugas menghandle permulaan eleksi dan `main` itu sendiri yang berisi kode apa yang harus dilakukan ketika ada pesan yang datang. Berikut penjelasan dari isi masing-masing:

- **startElection**
  start election berfungsi untuk memulai sebuah eleksi. Pertama, fungsi ini akan mengirimkan pesan “ELECTION” ke semua node dengan ID yang lebih tinggi. lalu, ia akan emnunggu respons. Jika ada **SATU** saja response “OK” diterima, maka dia akan menghentikan proses eleksi karena node tersebut bisa menyimpulkan bahwa bukan ID tertinggi. Ketika kondisi sebaliknya, dimana tidak ada pesan `"OK"` diterima, maka node tersebut akan langsung mengirimkan pesan `"COORDINATOR"` ke semua node yang ada.

```go
func startElection(nodeID int) {
        got_OK := false
        for i := nodeID + 1; i <= MAXNODEID; i++ {
                // EXPLANATION::: lets try to send to the higher nodes, is
there any?
        portStr := fmt.Sprintf("%d", 5000+i)
                repl, err := sendToNode("127.0.0.1", portStr, NODEID,
"ELECTION")
                if repl == "OK" && err == nil {
```

```
                        got_OK = true;
                }
        }

        if got_OK {
                // EXPLANATION::: got OK, so, there are/is higher node ID than
me, ok then
                return
        } else {

                // if there is no another OK, then this node is the highest
node
                for i := 0; i <= MAXNODEID; i++ {
                        if i == NODEID {
                                continue
                        }
                        repl, _ := sendToNode("127.0.0.1", fmt.Sprintf("%d",
5000+i), NODEID, fmt.Sprintf("COORDINATOR:%d", NODEID))
                        if repl == "OK" {
                                got_OK = true;
                        }
                }
        }

}
```

- **main (listener)**
  tentunya server juga harus memiliki sebuah listener yang mendengarkan incoming message. Di sini, saya menuliskan logika listener tersebut di dalam fungsi yang terdapat pada fungsi main yang dijalankan sebagai `goroutine` . fungsi ini menghandle 2 pesan utama, yaitu: \
  1. `ELECTION`
     ketika sebuah node mendapat pesan ini, maka, dia akan mengecek apakah node pengirim itu memiliki ID yang lebih kecil atau ID lebih besar dari penerima. Jika lebih kecil, node penerima akan memulai eleksi baru untuk mendapat node ID yang lebih besar. Sebaliknya, jika pengirim memiliki ID node lebih besar, maka penerima akan berhenti karena ada node dengan ID yang lebih besar yang lebih memungkinkan menjadi coordinator. \
  2. `COORDINATOR`
     ketika pesan ini didapatkan, maka tugas dari penerima adalah 1, mengubah tujuan koordinator mereka dengan koordinator baru yang telah memenangkan eleksi.

```go
    case "ELECTION":
        if node_from < NODEID {
```

```
            // EXPLANATION::: the node id where the message from is lower than
me
            sock.Send([]byte("OK"))

            // EXPLANATION::: so, send the ok, then let's see is there any
higher ID node?
            go startElection(NODEID) // Kick off your election routine
        } else {
            // EXPLANATION::: the node id where the message from is higher than
me
            // EXPLANATION::: just need to send the ok, because I will never be
the COORDINATOR
            sock.Send([]byte("OK"))
        }

    case "COORDINATOR":
        // EXPLANATION::: the new coordinator has been elected, so, let's
update the data.
        new_coordinator_int, _ := strconv.Atoi(actual_msg[1])
        CURRENTCOORDINATOR = new_coordinator_int
        nodePrintf("NEW COORDINATOR: %d", new_coordinator_int)
        sock.Send([]byte("OK"))
```

Selain kedua fungsi utama tersebut, terdapat fungsi-fungsi lain yang digunakan sebagai pendukung untuk menjalankan program ini seperti `sendToNode` dan `parseMessage`.

run.sh

Run.sh disesuaikan dengan ketentuans oal, sisinya adalah sebagai berikut:

```sh
./bully 0 live &
./bully 1 live &
./bully 2 live &
./bully 3 live &
./bully 4 live_init &
./bully 5 live &
./bully 6 live &
./bully 7 dead
```

Hasil

```
---------- Tahap initialisasi ----------

[ node 1] Starting up with state: live
[ node 1] Node listening on 127.0.0.1:5001...
[ node 0] Starting up with state: live
[ node 0] Node listening on 127.0.0.1:5000...
[ node 2] Starting up with state: live
[ node 2] Node listening on 127.0.0.1:5002...
[ node 6] Starting up with state: live
[ node 6] Node listening on 127.0.0.1:5006...
[ node 3] Starting up with state: live
[ node 3] Node listening on 127.0.0.1:5003...
[ node 5] Starting up with state: live
[ node 5] Node listening on 127.0.0.1:5005...
[ node 4] Starting up with state: live_init
[ node 7] Starting up with state: dead


---- Node 4 Memulai eleksi awal  ----

[ node 5] Received message: 4:ELECTION

---- Node 5 Memulai eleksi juga  ----

[ node 4] Got reply from 5: OK
[ node 6] Received message: 4:ELECTION
[ node 6] Received message: 5:ELECTION

---- Node 6 Memulai eleksi juga  ----

[ node 4] Got reply from 6: OK
[ node 4] Node listening on 127.0.0.1:5004...
[ node 5] Got reply from 6: OK

---- Node 6 tak mendapat respon OK  ----
--- Dia (Node 6) menjadi koordinator  ---
-- Node 6 broadcast jika ia koordinator  --
[ node 0] Received message: 6:COORDINATOR:6
[ node 0] NEW COORDINATOR: 6
[ node 0] Received message: 6:COORDINATOR:6
[ node 0] NEW COORDINATOR: 6
[ node 6] Got reply from 0: OK
[ node 6] Got reply from 0: OK
[ node 1] Received message: 6:COORDINATOR:6
[ node 1] NEW COORDINATOR: 6
[ node 1] Received message: 6:COORDINATOR:6
```

```
[ node 1] NEW COORDINATOR: 6
[ node 6] Got reply from 1: OK
[ node 6] Got reply from 1: OK
[ node 2] Received message: 6:COORDINATOR:6
[ node 2] NEW COORDINATOR: 6
[ node 2] Received message: 6:COORDINATOR:6
[ node 2] NEW COORDINATOR: 6
[ node 6] Got reply from 2: OK
[ node 6] Got reply from 2: OK
[ node 3] Received message: 6:COORDINATOR:6
[ node 3] NEW COORDINATOR: 6
[ node 3] Received message: 6:COORDINATOR:6
[ node 3] NEW COORDINATOR: 6
[ node 6] Got reply from 3: OK
[ node 6] Got reply from 3: OK
[ node 4] Received message: 6:COORDINATOR:6
[ node 4] NEW COORDINATOR: 6
[ node 4] Received message: 6:COORDINATOR:6
[ node 4] NEW COORDINATOR: 6
[ node 6] Got reply from 4: OK
[ node 6] Got reply from 4: OK
[ node 5] Received message: 6:COORDINATOR:6
[ node 5] NEW COORDINATOR: 6
[ node 6] Got reply from 5: OK
[ node 5] Received message: 6:COORDINATOR:6
[ node 5] NEW COORDINATOR: 6
[ node 6] Got reply from 5: OK

-- end --
```